TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC

BÁO CÁO THỰC HÀNH

Môn học: Kỹ thuật dữ liệu (Data Engineering)

Bài thực hành L03

Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

Tóm tắt **3**

1 Giới thiệu **3**

2 Cơ sở lý thuyết **4**
2.1 Kiến trúc Hive 4
2.2 Bảng ngoài, bảng quản lý và schema-on-read 4
2.3 Định dạng cột ORC và Parquet 4
2.4 Phân vùng và cắt tỉa phân vùng 4
2.5 Bucketing 4
2.6 Bảng giao dịch ACID 5

3 Môi trường và phương pháp thực nghiệm **5**
3.1 Môi trường 5
3.2 Dữ liệu 6
3.3 Quy trình 7

4 Kết quả thực nghiệm **7**
4.1 Bảng ngoài và nội dung Metastore 7
4.2 Bảng phân vùng ORC và Parquet 8
4.3 Kích thước lưu trữ theo định dạng 11
4.4 So sánh thời gian truy vấn CSV, ORC và Parquet 12
4.5 Cắt tỉa phân vùng 14
4.6 Bucketing 15
4.7 Phần mở rộng: thống kê và bảng ACID 16

5 Thảo luận **19**
5.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì? 19
5.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`? 19
5.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần? 19
5.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet? 19
5.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`? . . 20
5.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao? 20
5.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định? 20
5.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao? 21
5.9 Vì sao `count(*)` không nhanh hơn sau khi đã có thống kê? 21
5.10 Hạn chế của thực nghiệm 21

6 Kết luận **22**

Tài liệu tham khảo **22**

Phụ lục A: Danh mục tệp nộp kèm **23**

Phụ lục B: Trích đoạn mã nguồn **23**

Danh mục hình

1 Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS. 5
2 Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez. 6
3 Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL). 8
4 `SHOW PARTITIONS` và số chuyến theo `pickup_month`. 9
5 Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode. 10
6 Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode. . . . 10
7 `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024. . . . 11
8 Kích thước CSV, ORC và Parquet trên HDFS. 12
9 Lần chạy thứ hai của `03_compare_formats.sql`. 13
10 Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`. 15
11 Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK. . . 16
12 Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode. 16
13 Phần mở rộng: thống kê cột và các thao tác ACID. 18
14 Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode. 18

Danh mục bảng

1 Môi trường thực nghiệm. 5
2 Dữ liệu đầu vào. 6
3 Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài. 7
4 Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động. 8
5 Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng. 12
6 Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai). 12
7 Kết quả Q1 (giống nhau trên CSV, ORC và Parquet). 14
8 Kết quả Q2: số chuyến và doanh thu theo borough. 14
9 Kết quả kiểm chứng cắt tỉa phân vùng. 14
10 Các tệp bucket của `trips_bucketed`. 15

11 Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng. 17
12 Các dòng có thời gian đón khách nằm ngoài tháng 01/2024. 21
13 Danh mục tệp nộp kèm trong `submission/L03/`. 23

Tóm tắt

Bài thực hành dựng Apache Hive 4.2.1 trên HDFS, Metastore độc lập dùng PostgreSQL 18, Tez chạy ở chế độ cục bộ. Toàn bộ dữ liệu taxi vàng New York tháng 01/2024 (2,964,624 dòng, tệp CSV 274.2 MB) được đưa lên HDFS dưới dạng bảng ngoài, rồi chuyển sang hai bảng phân vùng ORC (nén ZLIB) và Parquet (nén Snappy) bằng phân vùng động. Chúng tôi đo kích thước lưu trữ và thời gian truy vấn, kiểm tra cắt tỉa phân vùng bằng `EXPLAIN`, và thử bucketing. ORC nhỏ hơn CSV 5.2 lần, Parquet nhỏ hơn 4.2 lần; truy vấn Q1 giảm từ 8.53 s trên CSV xuống 1.89 s (ORC) và 1.62 s (Parquet). Lọc theo cột phân vùng chỉ đọc 1 trên 5 phân vùng, còn lọc theo cột thời gian gốc phải đọc cả 5. Bảng bucketing có đúng 4 tệp, và toàn bộ 145,240 chuyến từ JFK nằm trong một tệp. Phân vùng động cũng làm lộ ra 18 dòng có thời gian nằm ngoài tháng, một lỗi chất lượng dữ liệu lẽ ra phải chặn từ trước khi ghi.

Từ khóa: Apache Hive, Metastore, ORC, Parquet, partition pruning, bucketing, ACID, Tez.

1 Giới thiệu

Hive là hệ thống đầu tiên đưa SQL lên Hadoop [1]. Ý tưởng chính của nó là tách siêu dữ liệu (lược đồ, vị trí, phân vùng) khỏi dữ liệu: siêu dữ liệu nằm trong một cơ sở dữ liệu quan hệ gọi là Metastore, còn dữ liệu là các tệp bình thường trên HDFS. Cách làm này về sau thành chuẩn chung. Ngày nay Spark SQL, Trino, Impala và cả bảng Iceberg vẫn dùng Metastore của Hive làm *catalog* (danh mục bảng), nên hiểu Hive tổ chức tệp và phân vùng ra sao vẫn còn giá trị.

Các mục tiêu của bài:

1. Mô tả kiến trúc Hive: HiveServer2, Metastore, Tez và HDFS.
2. Tạo bảng ngoài trên tệp có sẵn với `LazySimpleSerDe` và `OpenCSVSerde`.
3. Tạo bảng phân vùng ORC và Parquet, nạp dữ liệu bằng phân vùng động.
4. Đo ảnh hưởng của định dạng tệp (CSV, ORC, Parquet) lên kích thước và thời gian truy vấn.
5. Kiểm chứng cắt tỉa phân vùng bằng `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.
6. Tạo bảng bucketing, lấy mẫu theo bucket; phần mở rộng thử bảng ACID và thống kê.

Lý thuyết ở Mục 2, cách làm ở Mục 3, kết quả ở Mục 4, các câu hỏi checkpoint ở Mục 5.

2 Cơ sở lý thuyết

2.1 Kiến trúc Hive

Máy khách (Beeline, JDBC) gửi câu lệnh tới HiveServer2 ở cổng 10000. HiveServer2 phân tích cú pháp, tối ưu kế hoạch bằng Calcite (CBO) rồi sinh một DAG Tez [6]. Lúc lập kế hoạch, nó hỏi Metastore (Thrift, cổng 9083) về lược đồ, định dạng và vị trí của từng bảng, từng phân vùng; Metastore giữ các thông tin này trong PostgreSQL ở các bảng `TBLS`, `SDS`, `PARTITIONS`. Các task Tez sau đó đọc và ghi tệp trên HDFS. Ở môi trường thật, task Tez chạy trong container YARN hoặc LLAP. Trong bài này Tez chạy cục bộ ngay trong HiveServer2, nhưng câu SQL, kế hoạch và tệp sinh ra không khác gì.

2.2 Bảng ngoài, bảng quản lý và schema-on-read

Hive theo kiểu *schema-on-read* (áp lược đồ lúc đọc): lược đồ chỉ dùng để diễn giải tệp khi truy vấn. Bảng ngoài (`EXTERNAL TABLE`) chỉ là siêu dữ liệu trỏ tới một thư mục, nên `DROP TABLE` không xóa tệp. Hive 4 bắt bảng quản lý phải là bảng ACID; vì vậy một câu `CREATE TABLE ... STORED AS ORC` bình thường bị Metastore chuyển thành bảng ngoài trong warehouse kèm `external.table.purge=TRUE`, và khi đó `DROP TABLE` vẫn xóa tệp [2].

2.3 Định dạng cột ORC và Parquet

ORC [3] và Parquet [4] đặt các giá trị cùng cột liền nhau trong từng *stripe* (ORC) hay *row group* (Parquet). Nhờ vậy có thể dùng dictionary encoding, *run-length encoding* (RLE), mã hóa delta, bit-packing, rồi nén thêm bằng ZLIB hoặc Snappy. Mỗi khối có thống kê min/max để bộ đọc bỏ qua khối không thỏa điều kiện (*predicate pushdown*), và bộ đọc chỉ lấy những cột cần (*column pruning*) [5].

2.4 Phân vùng và cắt tỉa phân vùng

Partition chia dữ liệu thành các thư mục con dạng `cot=gia_tri/`. Khi điều kiện lọc nằm trên cột phân vùng, trình lập kế hoạch chỉ liệt kê những thư mục khớp (*partition pruning*). Giá trị phân vùng có thể tĩnh (ghi trong câu lệnh) hoặc động (lấy từ cột cuối của `SELECT`). Mặc định Hive ở chế độ `strict`, bắt phải có ít nhất một giá trị tĩnh [2].

2.5 Bucketing

Bucketing chia dữ liệu theo hash(c) mod N vào đúng N tệp, N cố định trong DDL. Kỹ thuật này hợp với cột có nhiều giá trị phân biệt, dùng được cho `TABLESAMPLE`, bucket map join và sort-merge-bucket join [2], [7].

2.6 Bảng giao dịch ACID

Bảng ACID không sửa tệp tại chỗ. Mỗi giao dịch ghi một thư mục `delta_*` (dòng mới) hoặc `delete_delta_*` (đánh dấu dòng bị xóa). Bộ đọc hợp nhất các delta lúc đọc, và *compaction* gộp chúng lại về sau [5].

3 Môi trường và phương pháp thực nghiệm

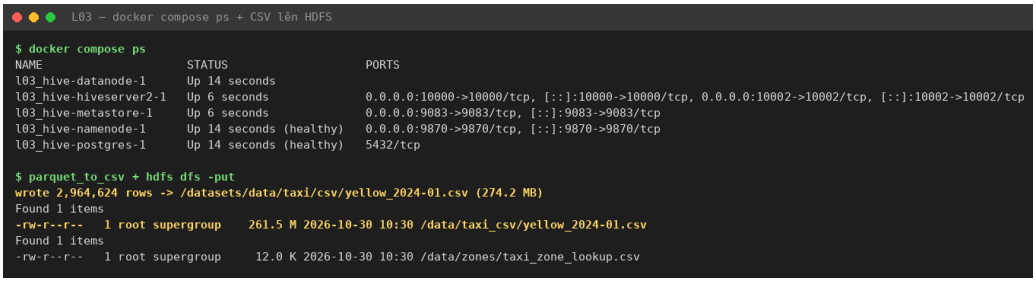
3.1 Môi trường

Cả cụm chạy bằng Docker Compose trên một máy (Bảng 1).

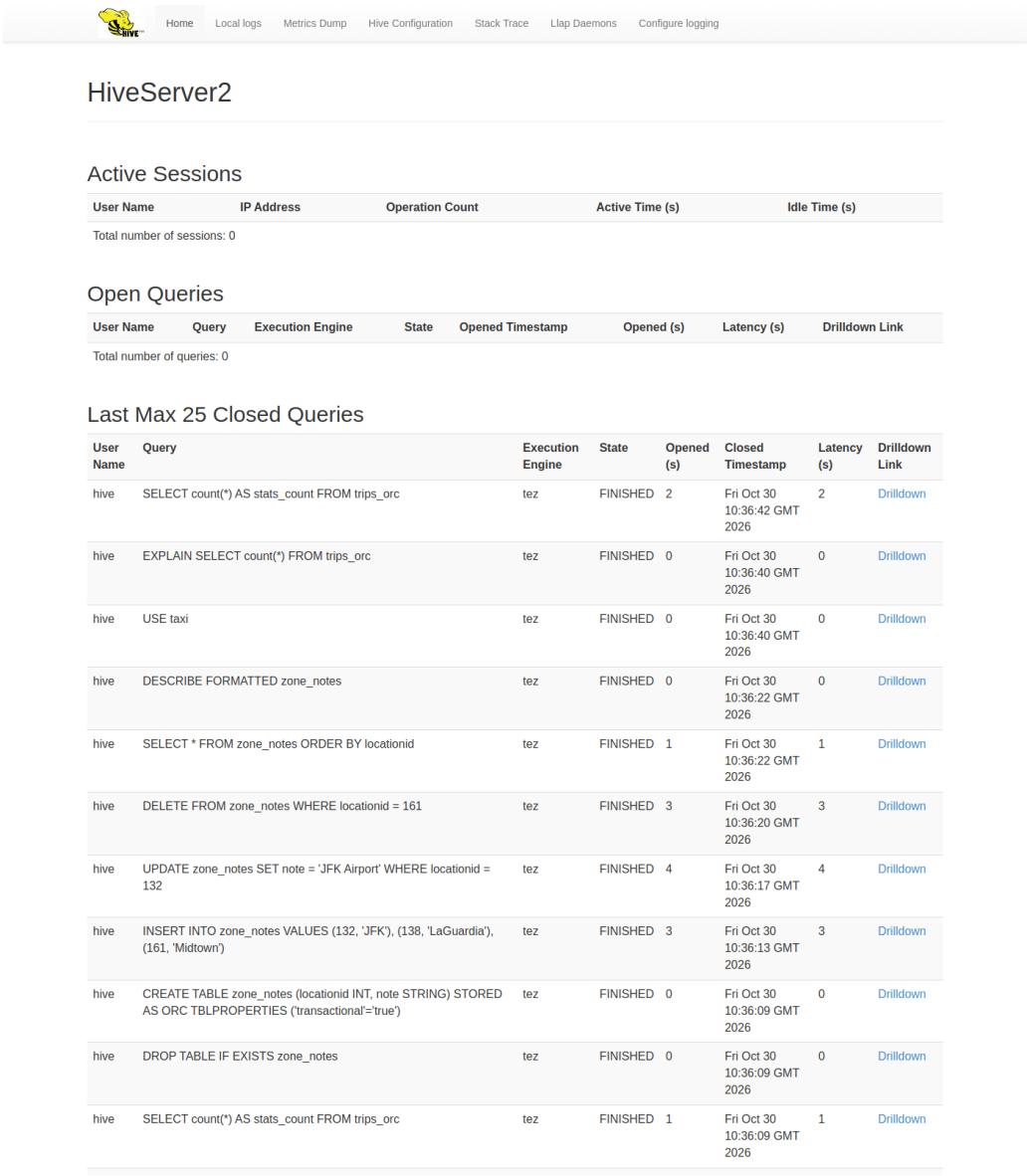
Bảng 1. Môi trường thực nghiệm.

Thành phần	Phiên bản / cấu hình
Hive	4.2.1 (image chính thức `apache/hive:4.2.1`), HiveServer2 + Metastore độc lập
Bộ máy thực thi	Tez, chế độ cục bộ bên trong HiveServer2
Cơ sở dữ liệu Metastore	PostgreSQL 18 (database `metastore`)
HDFS	3.4.3, 1 NameNode + 1 DataNode (hệ số nhân bản 1)
Cổng dịch vụ	HiveServer2 10000 (JDBC), 10002 (Web UI); Metastore 9083; NameNode UI 9870
Máy khách	Beeline qua `run_sql.sh` (in thời gian từng câu lệnh)

Năm container đều `Up`, NameNode và PostgreSQL ở trạng thái `healthy` (Hình 1). Giao diện web của HiveServer2 liệt kê các truy vấn đã chạy xong, tất cả trên `tez` (Hình 2).



Hình 1. Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS.



Home Local logs Metrics Dump Hive Configuration Stack Trace Llap Daemons Configure logging

HiveServer2

Active Sessions

User Name	IP Address	Operation Count	Active Time (s)	Idle Time (s)

Total number of sessions: 0

Open Queries

User Name	Query	Execution Engine	State	Opened Timestamp	Opened (s)	Latency (s)	Drilldown Link

Total number of queries: 0

Last Max 25 Closed Queries

User Name	Query	Execution Engine	State	Opened (s)	Closed Timestamp	Latency (s)	Drilldown Link
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	2	Fri Oct 30 10:36:42 GMT 2026	2	Drilldown
hive	EXPLAIN SELECT count(*) FROM trips_orc	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	USE taxi	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	DESCRIBE FORMATTED zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:22 GMT 2026	0	Drilldown
hive	SELECT * FROM zone_notes ORDER BY locationid	tez	FINISHED	1	Fri Oct 30 10:36:22 GMT 2026	1	Drilldown
hive	DELETE FROM zone_notes WHERE locationid = 161	tez	FINISHED	3	Fri Oct 30 10:36:20 GMT 2026	3	Drilldown
hive	UPDATE zone_notes SET note = 'JFK Airport' WHERE locationid = 132	tez	FINISHED	4	Fri Oct 30 10:36:17 GMT 2026	4	Drilldown
hive	INSERT INTO zone_notes VALUES (132, 'JFK'), (138, 'LaGuardia'), (161, 'Midtown')	tez	FINISHED	3	Fri Oct 30 10:36:13 GMT 2026	3	Drilldown
hive	CREATE TABLE zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	DROP TABLE IF EXISTS zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	1	Fri Oct 30 10:36:09 GMT 2026	1	Drilldown

Hình 2. Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez.

3.2 Dữ liệu

Tệp `yellow_tripdata_2024-01.parquet` được chuyển sang CSV không có dòng tiêu đề (Bảng 2). Đề bài gợi ý dùng mẫu 1 triệu dòng, nhưng chúng tôi dùng nguyên tháng, nên các con số trong bài lớn hơn số tham khảo.

Bảng 2. Dữ liệu đầu vào.

Tệp trên HDFS	Kích thước	Số dòng
`/data/taxi_csv/yellow_2024-01.csv`	274,217,811 B (274.2 MB, 261.5 MiB)	2,964,624
`/data/zones/taxi_zone_lookup.csv`	12.0 K	265 zone + 1 dòng tiêu đề

3.3 Quy trình

Mỗi bước chạy bằng `run_sql.sh`, log gốc lưu trong các tệp `*.txt`:

1. Khởi động cụm, chuyển Parquet sang CSV, tải lên `/data/taxi_csv` và `/data/zones`.
2. `01_external_tables.sql`: tạo bảng ngoài `trips_csv` (`ROW FORMAT DELIMITED`) và `zones` (`OpenCSVSerde`), rồi truy vấn thẳng bảng `TBLS`, `SDS` của Metastore trong PostgreSQL.
3. `02_partitioned_tables.sql`: bật `hive.exec.dynamic.partition.mode=nonstrict`, tính `pickup_month` bằng `date_format(..., 'yyyy-MM')`, tạo và nạp `trips_orc` (ZLIB) và `trips_parquet` (Snappy). Đo kích thước bằng `hdfs dfs -du`.
4. `03_compare_formats.sql`, chạy hai lần với `hive.compute.query.using.stats=false`, `hive.query.results.cache.enabled=false` và `hive.fetch.task.conversion=none` để Hive phải quét thật. Chúng tôi lấy số của lần thứ hai, khi JVM và Tez đã khởi động xong.
5. `04_partition_pruning.sql`: thêm một `EXPLAIN DEPENDENCY` lọc theo `tpep_pickup_datetime` để đối chiếu.
6. `05_bucketing.sql`: bảng `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`.
7. Phần mở rộng: `ANALYZE TABLE` cho thống kê cột và bảng ACID `zone_notes`.

4 Kết quả thực nghiệm

4.1 Bảng ngoài và nội dung Metastore

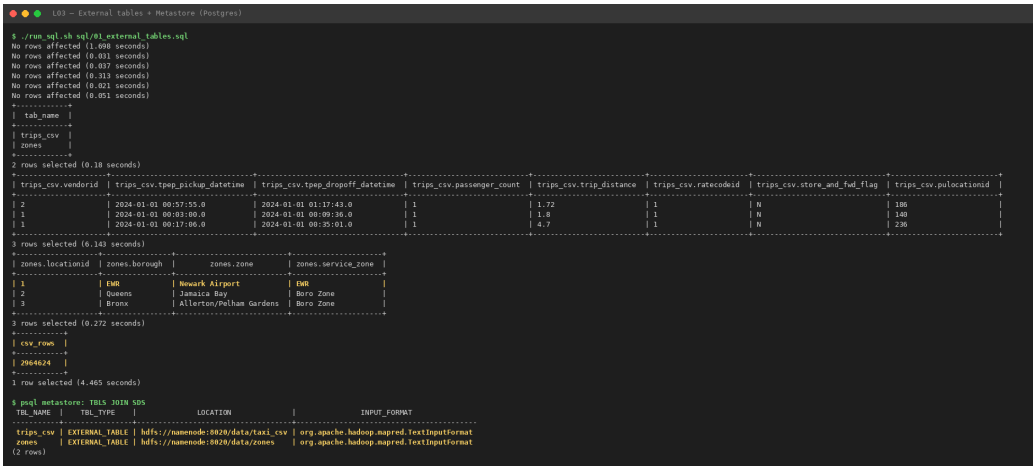
Cả hai bảng ngoài đọc được ngay. `SELECT * FROM zones LIMIT 3` trả về `1 | EWR | Newark Airport | EWR`, `2 | Queens | Jamaica Bay | Boro Zone` và `3 | Bronx | Allerton/Pelham Gardens | Boro Zone`, tức `OpenCSVSerde` đã bỏ dòng tiêu đề và dấu ngoặc kép. `count(*)` trên `trips_csv` ra `csv_rows = 2,964,624` sau 4.465 s, khớp số dòng của tệp.

Truy vấn `TBLS JOIN SDS` trong PostgreSQL (Bảng 3, Hình 3) chỉ thấy tên bảng, loại bảng, vị trí và lớp định dạng đầu vào. Không có dòng dữ liệu nào trong Metastore.

Bảng 3. Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài.

TBL_NAME	TBL_TYPE	LOCATION	INPUT_FORMAT
trips_csv	EXTERNAL_TABLE	`hdfs://namenode:8020/data/taxi_csv`	`org.apache.hadoop.mapred.TextInputFormat`

TBL_NAME	TBL_TYPE	LOCATION	INPUT_FORMAT
zones	EXTERNAL_TABLE	`hdfs://namenode:8020/data/zones`	`org.apache.hadoop.mapred.TextInputFormat`



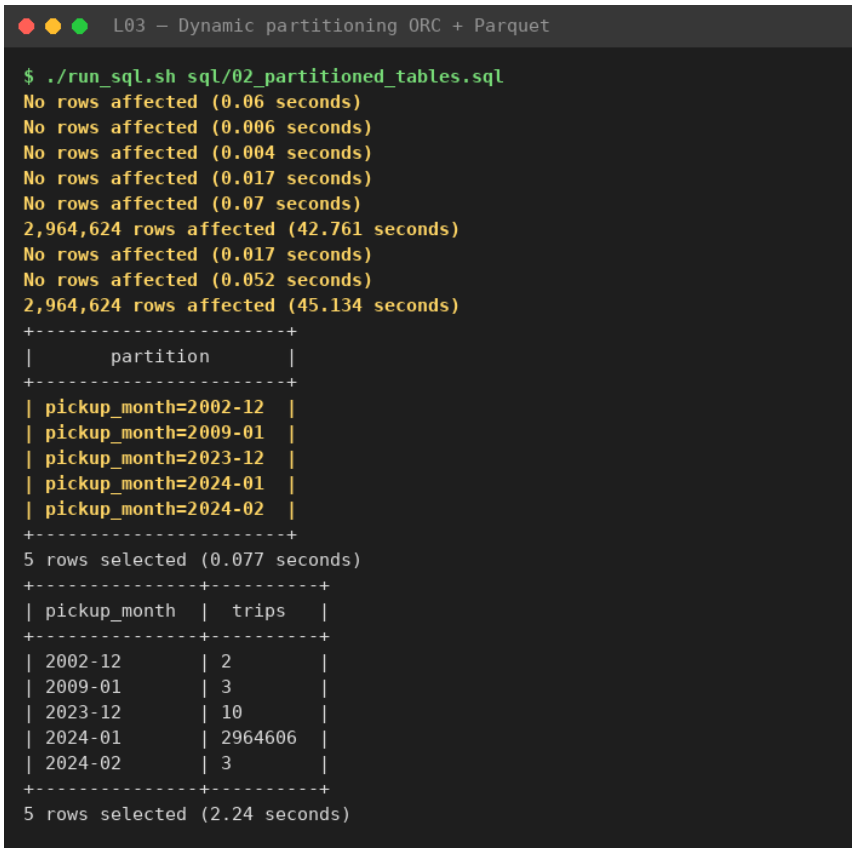
Hình 3. Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL).

4.2 Bảng phân vùng ORC và Parquet

`INSERT OVERWRITE` ghi 2,964,624 dòng vào `trips_orc` trong 42.761 s và vào `trips_parquet` trong 45.134 s. Chúng tôi chờ 4 phân vùng như kết quả mẫu của đề, nhưng `SHOW PARTITIONS` trả về 5 (Bảng 4, Hình 4): dữ liệu cả tháng có thêm phân vùng `2024-02`.

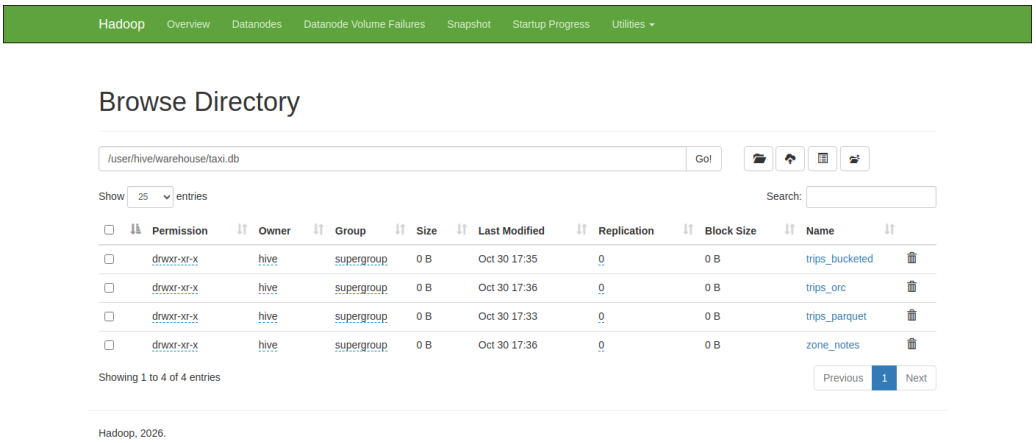
Bảng 4. Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động.

pickup_month	Số chuyến	Tệp ORC	Kích thước tệp
2002-12	2	`000007_0`	1,637 B
2009-01	3	`000004_0`	1,922 B
2023-12	10	`000001_0`	2,234 B
2024-01	2,964,606	`000005_0`	52,621,888 B
2024-02	3	`000005_0`	1,993 B

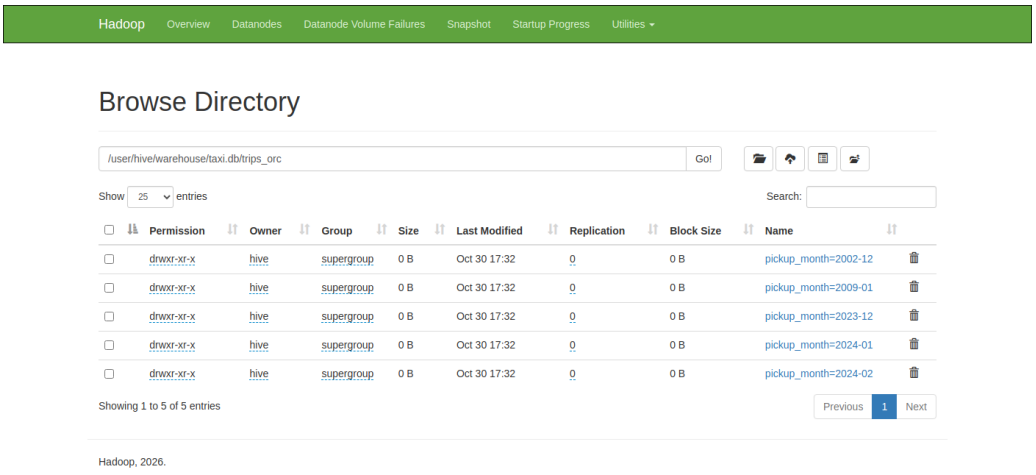


Hình 4. `SHOW PARTITIONS` và số chuyến theo `pickup_month`.

Trên giao diện NameNode, thư mục `taxi.db` có bốn bảng `trips_bucketed`, `trips_orc`, `trips_parquet`, `zone_notes` (Hình 5), và `trips_orc` có năm thư mục con `pickup_month=...` (Hình 6). Bốn phân vùng nhỏ chỉ có một tệp khoảng 2 KB, một phiên bản thu nhỏ của vấn đề tệp nhỏ ở Mục 5.6.

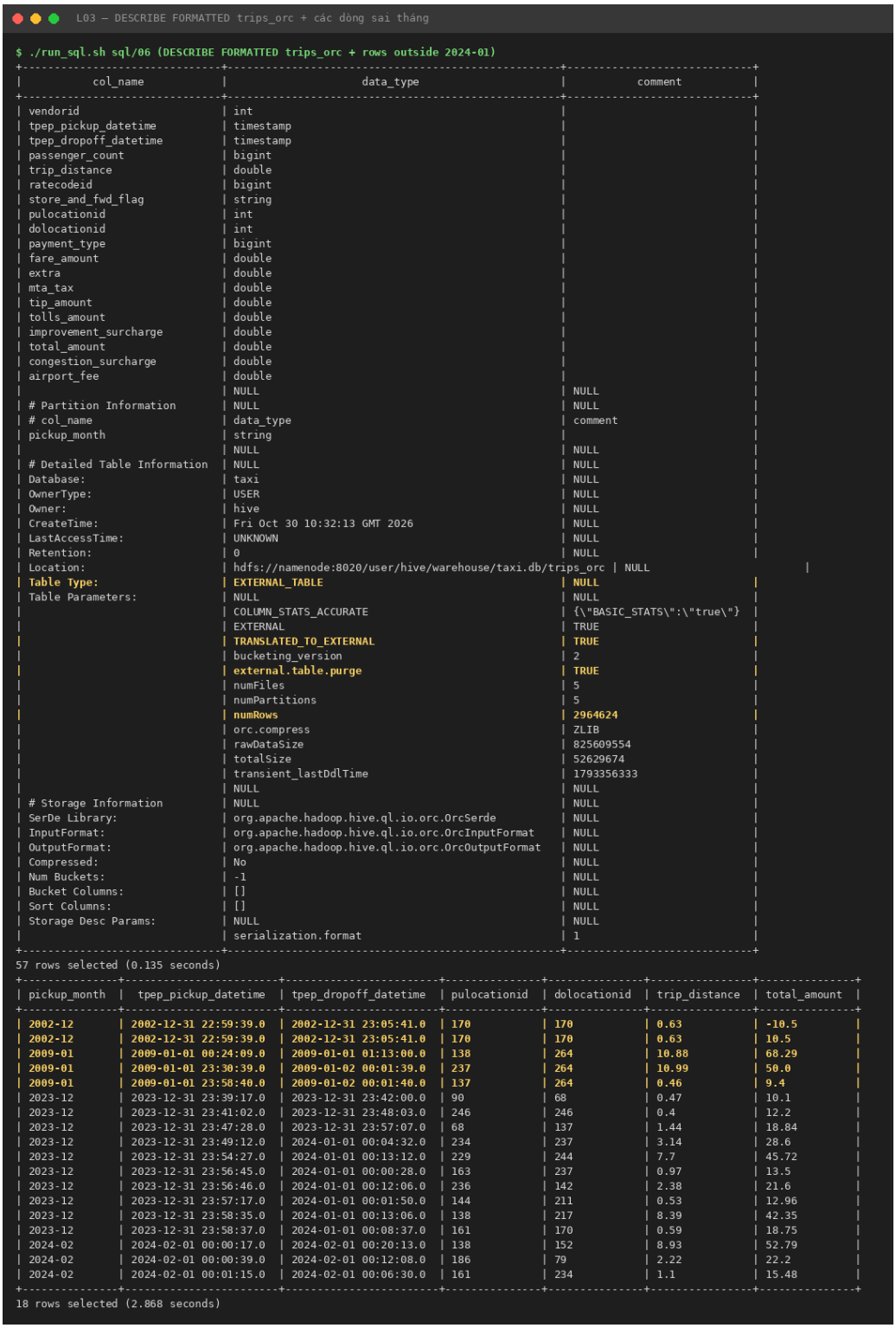


Hình 5. Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode.



Hình 6. Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode.

`DESCRIBE FORMATTED trips_orc` (Hình 7) ghi `Table Type: EXTERNAL_TABLE` cùng `TRANSLATED_TO_EXTERNAL=TRUE` và `external.table.purge=TRUE`, đúng cách Hive 4 xử lý bảng "managed" không phải ACID. Thống kê trong Metastore: `numFiles=5`, `numPartitions=5`, `numRows=2964624`, `rawDataSize=825609554`, `totalSize=52629674`. Hình này cũng liệt kê 18 dòng có `pickup_month` khác `2024-01`, bàn ở Mục 5.8.



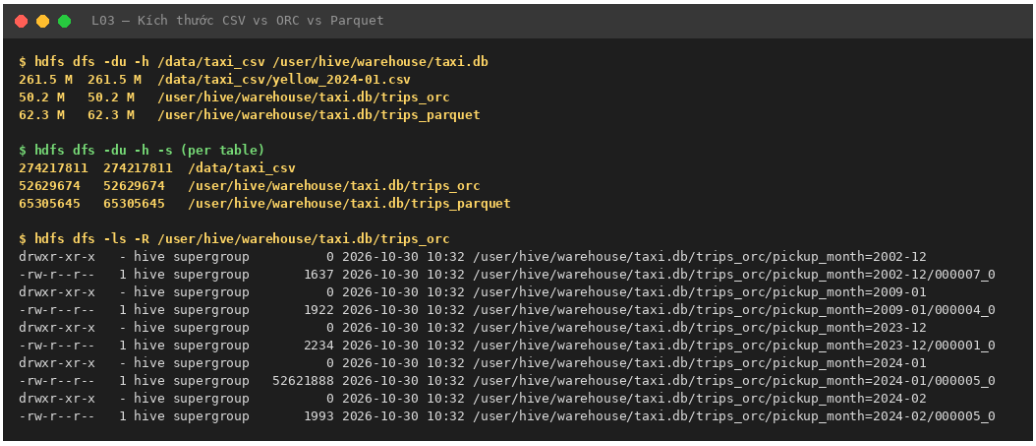
Hình 7. `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024.

4.3 Kích thước lưu trữ theo định dạng

Kích thước đo bằng `hdfs dfs -du -s` nằm ở Bảng 5 và Hình 8. Tỉ lệ nén tính bằng $r = S_{\text{CSV}}/S_{\text{định dạng}}$, ví dụ $274{,}217{,}811/52{,}629{,}674 \approx 5.2$.

Bảng 5. Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng.

Định dạng	Kích thước (byte)	Kích thước (MiB)	Nhỏ hơn CSV
CSV (text)	274,217,811	261.5	1×
ORC + ZLIB	52,629,674	50.2	**5.2×**
Parquet + Snappy	65,305,645	62.3	4.2×
ORC bucketed (chỉ 5 cột)	25,271,072	24.1	10.9×



Hình 8. Kích thước CSV, ORC và Parquet trên HDFS.

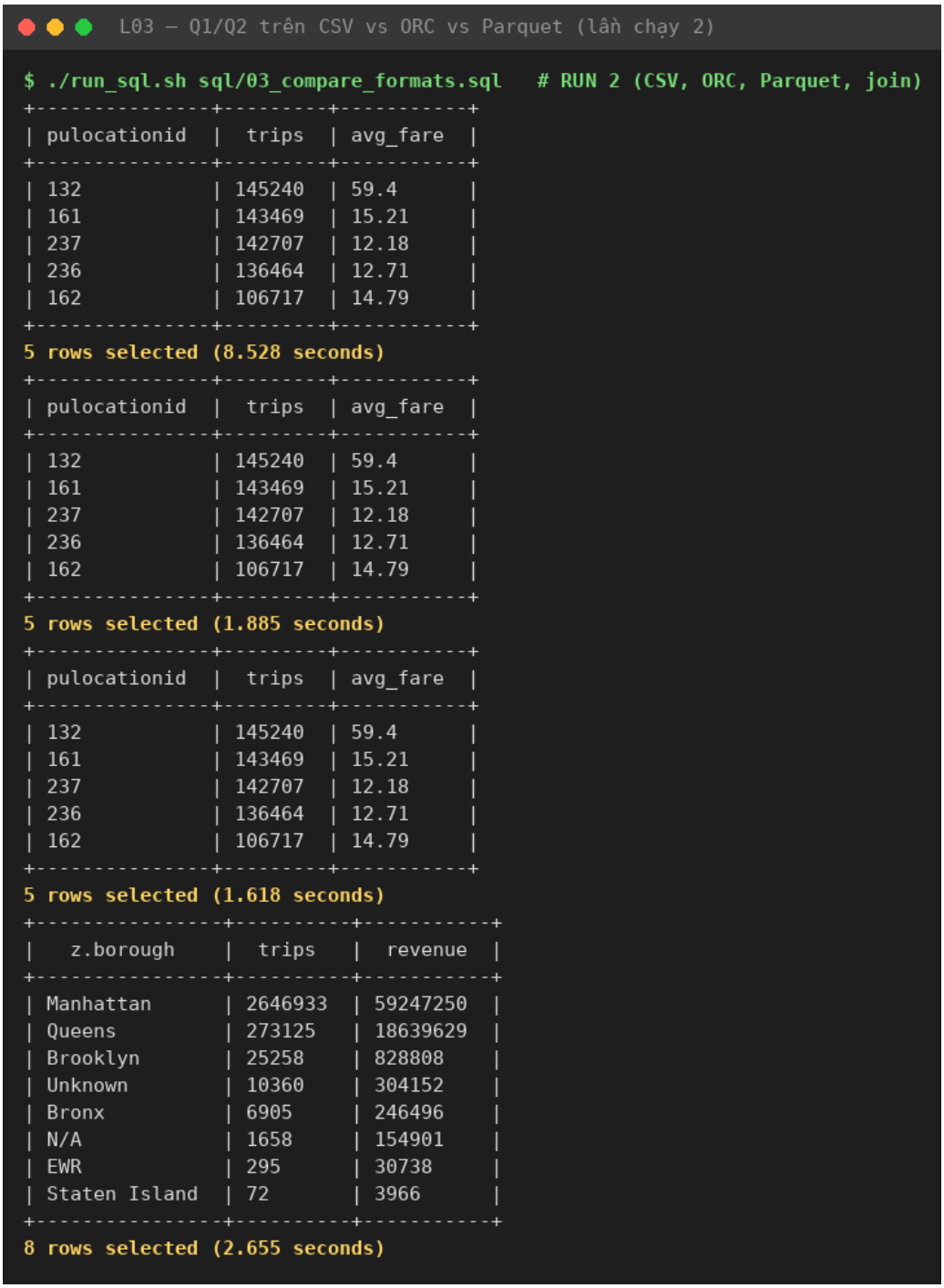
ORC + ZLIB nhỏ hơn Parquet + Snappy khoảng 19 %. Điều này hợp lý vì ZLIB thiên về tỉ lệ nén, Snappy thiên về tốc độ. Bảng bucketed chỉ giữ 5 trên 19 cột nên con số 10.9× không đem so với ba dòng trên được.

4.4 So sánh thời gian truy vấn CSV, ORC và Parquet

Q1 tìm năm zone đón khách nhiều nhất trong tháng 01/2024, chỉ dùng `pulocationid`, `fare_amount` và điều kiện thời gian. Trên CSV, điều kiện là `tpep_pickup_datetime`; trên ORC và Parquet là `pickup_month = '2024-01'`. Q2 nối `trips_orc` với `zones`. Thời gian cả hai lần chạy ở Bảng 6, ảnh lần chạy thứ hai ở Hình 9.

Bảng 6. Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai).

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q1 top 5 zone đón khách	1	9.222 s	1.987 s	1.767 s
Q1 top 5 zone đón khách	2	8.528 s	1.885 s	1.618 s
Q2 nối `zones` theo borough	1		2.939 s	
Q2 nối `zones` theo borough	2		2.655 s	



Hình 9. Lần chạy thứ hai của `03_compare_formats.sql`.

Ở lần thứ hai, ORC nhanh hơn CSV khoảng $8.528/1.885 \approx 4.5$ lần, Parquet khoảng $8.528/1.618 \approx 5.3$ lần. Ba nguồn cho cùng một kết quả (Bảng 7), nên việc đổi định dạng không làm sai dữ liệu. Zone `132` (sân bay JFK) đứng đầu, và giá cước trung bình ở đây cao gấp mấy lần các zone trong nội đô.

Bảng 7. Kết quả Q1 (giống nhau trên CSV, ORC và Parquet).

pulocationid	Số chuyến	Giá cước trung bình
132	145,240	59.40
161	143,469	15.21
237	142,707	12.18
236	136,464	12.71
162	106,717	14.79

Theo Q2 (Bảng 8), Manhattan chiếm 89.3 % số chuyến. Cộng số chuyến các borough lại ra đúng 2,964,606, bằng số dòng của phân vùng `2024-01`.

Bảng 8. Kết quả Q2: số chuyến và doanh thu theo borough.

Borough	Số chuyến	Doanh thu (`total_amount`)
Manhattan	2,646,933	59,247,250
Queens	273,125	18,639,629
Brooklyn	25,258	828,808
Unknown	10,360	304,152
Bronx	6,905	246,496
N/A	1,658	154,901
EWR	295	30,738
Staten Island	72	3,966

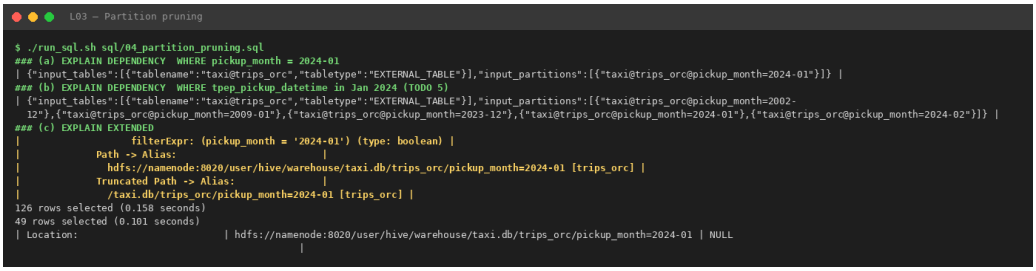
4.5 Cắt tỉa phân vùng

Ba câu `EXPLAIN` được đặt cạnh nhau ở Bảng 9 và Hình 10.

Bảng 9. Kết quả kiểm chứng cắt tỉa phân vùng.

Thí nghiệm	Điều kiện lọc	Kết quả
(a) `EXPLAIN DEPENDENCY`	`pickup_month = '2024-01'`	`input_partitions` có **1** phân vùng (`pickup_month=2024-01`)
(b) `EXPLAIN DEPENDENCY`	`tpep_pickup_datetime` trong tháng 1	`input_partitions` có cả 5 phân vùng

Thí nghiệm	Điều kiện lọc	Kết quả
(c) `EXPLAIN EXTENDED`	`pickup_month = '2024-01'`	`filterExpr: (pickup_month = '2024-01')`; `Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`



Hình 10. Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.

Trong kế hoạch (c), `TableScan` ước lượng `Num rows: 2964606`, `Data size: 849321344`, `Basic stats: COMPLETE`, đúng bằng số dòng của riêng phân vùng `2024-01`. Cả `Map 1` và `Reducer 2` đều chạy `Execution mode: vectorized`. Ở (b), Hive không biết `tpep_pickup_datetime` và `pickup_month` có liên quan với nhau, vì trong lược đồ đó là hai cột độc lập. Nó phải liệt kê mọi phân vùng rồi lọc từng dòng.

4.6 Bucketing

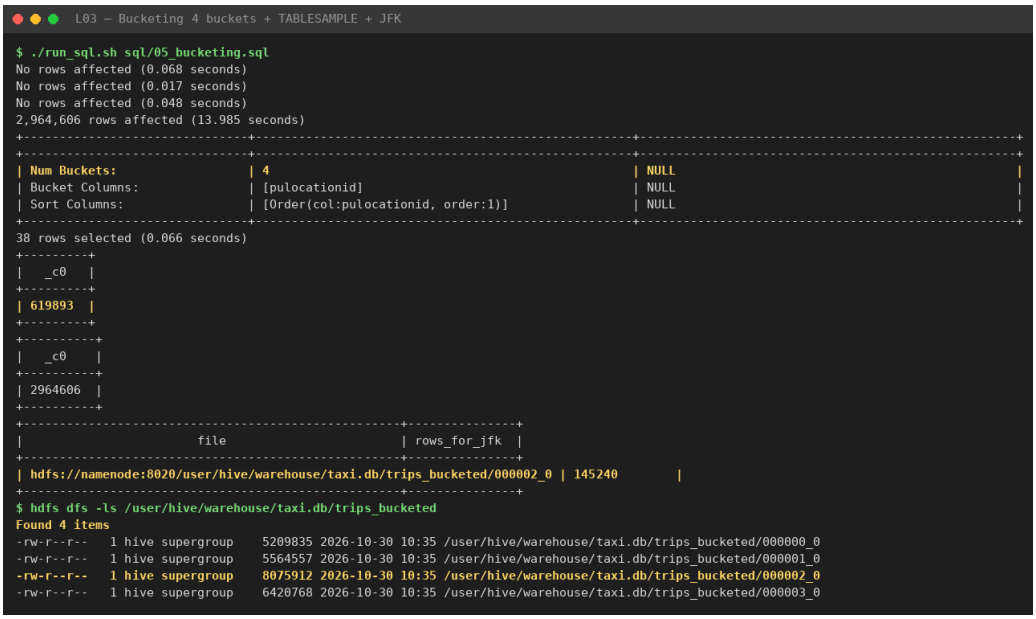
Bảng `trips_bucketed` (5 cột) nạp 2,964,606 dòng từ phân vùng `2024-01` trong 13.985 s. `DESCRIBE FORMATTED` ghi `Num Buckets: 4`, `Bucket Columns: [pulocationid]` và `Sort Columns: [Order(col:pulocationid, order:1)]`. Thư mục bảng có đúng bốn tệp (Bảng 10, Hình 12).

Bảng 10. Các tệp bucket của `trips_bucketed`.

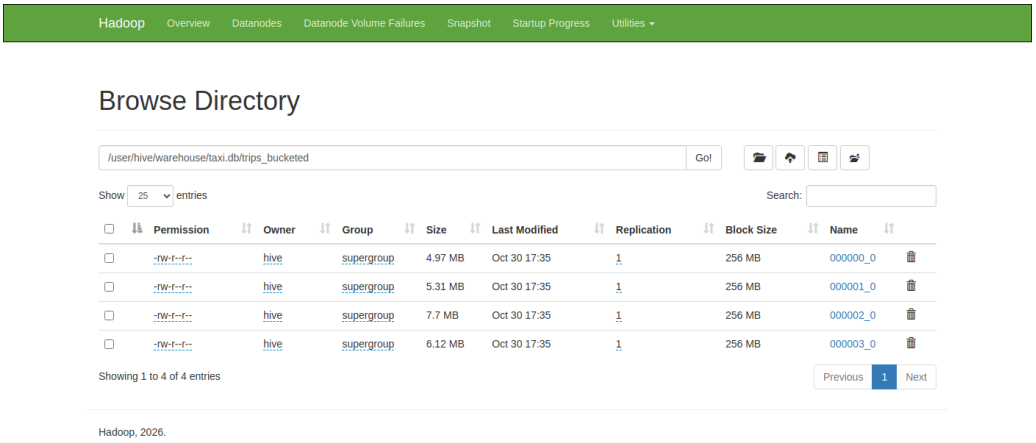
Tệp	Kích thước	Ghi chú
`000000_0`	5,209,835 B	
`000001_0`	5,564,557 B	
`000002_0`	8,075,912 B	chứa toàn bộ 145,240 dòng JFK (zone 132)
`000003_0`	6,420,768 B	
Tổng	25,271,072 B	`numFiles=4`, `numRows=2964606`

`TABLESAMPLE(BUCKET 1 OUT OF 4 ON pulocationid)` trả về 619,893 dòng sau 1.606 s, tức khoảng 20.9 % chứ không phải 25 %. Lý do là hàm băm chia theo giá trị khóa chứ không theo

số dòng; bucket nào nhận phải các zone đông khách sẽ to hơn. Truy vấn `INPUT__FILE__NAME` cho zone 132 chỉ trả về một tệp `000002_0` (Hình 11), nên tệp này cũng là tệp lớn nhất.



Hình 11. Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK.



Hình 12. Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode.

4.7 Phần mở rộng: thống kê và bảng ACID

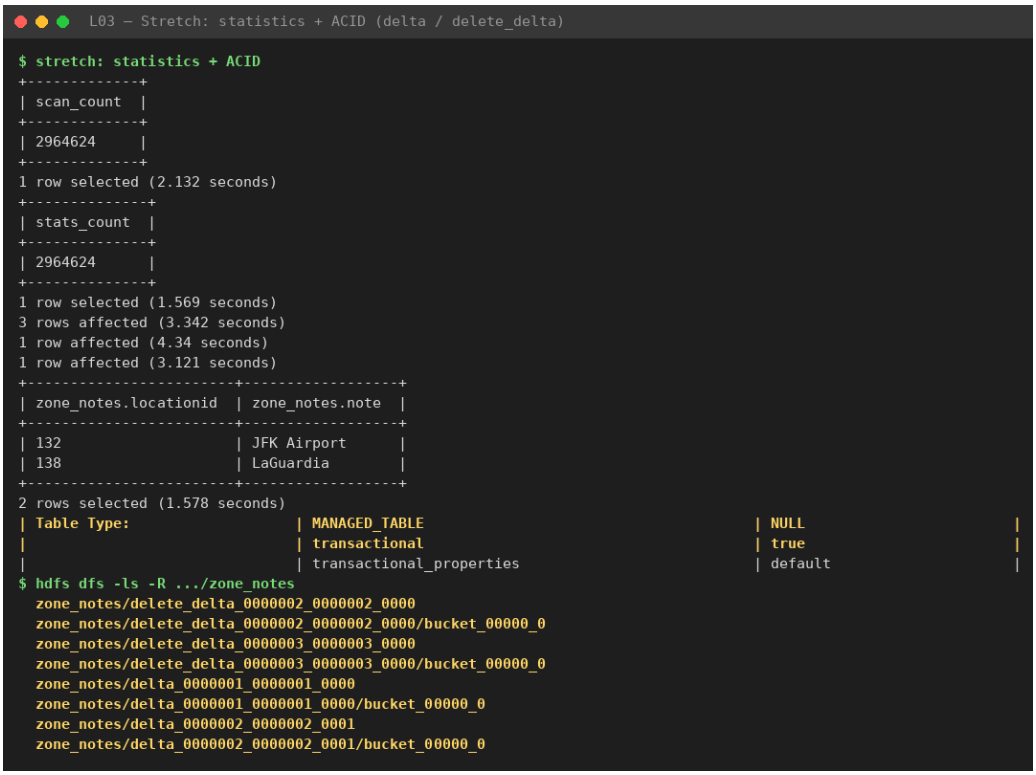
Trước khi có thống kê, `SELECT count(*)` trên `trips_orc` mất 2.132 s. Câu `ANALYZE TABLE trips_orc PARTITION (pickup_month) COMPUTE STATISTICS FOR COLUMNS` mất 18.354 s. Chúng tôi chờ `count(*)` sau đó sẽ trả lời ngay từ Metastore khi bật `hive.compute.query.`

`using.stats=true`, nhưng không phải vậy: kết quả vẫn là 2,964,624 và vẫn chạy DAG Tez, mất 1.569 s lần đầu và 2.592 s khi đo lại. `EXPLAIN` lần đo lại vẫn có `TableScan [TS_0] (rows= 2964624 width=290)` với `Tbl:COMPLETE, Col:COMPLETE`. Thống kê đã đủ mà không được dùng; nguyên nhân ở Mục 5.9.

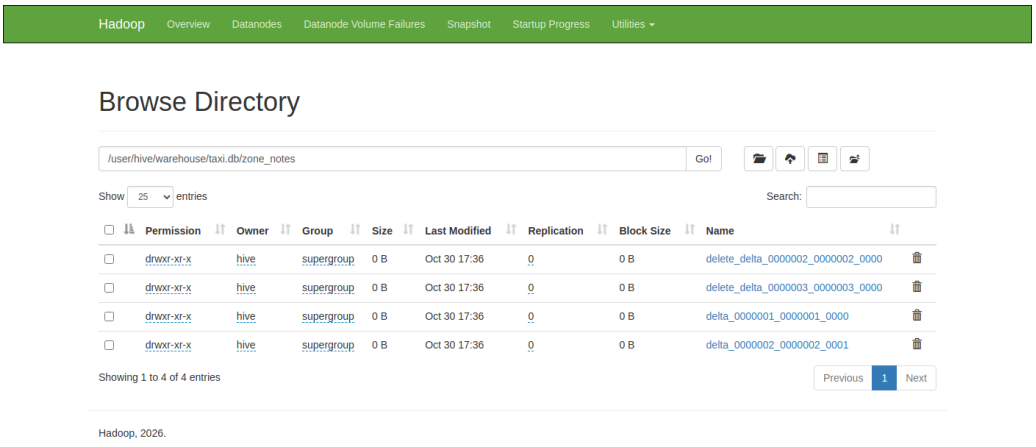
Bảng `zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')` mới là `MANAGED_TABLE` thật. Các thao tác và thư mục sinh ra trên HDFS nằm ở Bảng 11, Hình 13 và Hình 14. Cuối cùng còn 2 dòng: `132 | JFK Airport` và `138 | LaGuardia`.

Bảng 11. Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng.

Giao dịch	Câu lệnh	Thời gian	Thư mục sinh ra
1	`INSERT` (132 JFK), (138 LaGuardia), (161 Midtown)	3.342 s (3 dòng)	`delta_0000001_ 0000001_0000`
2	`UPDATE ... SET note='JFK Airport' WHERE locationid=132`	4.34 s (1 dòng)	`delete_delta_ 0000002_0000002_ 0000` + `delta_ 0000002_0000002_ 0001`
3	`DELETE ... WHERE locationid=161`	3.121 s (1 dòng)	`delete_delta_ 0000003_0000003_ 0000`



Hình 13. Phần mở rộng: thống kê cột và các thao tác ACID.



Hình 14. Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode.

Không tệp nào bị sửa tại chỗ. `UPDATE` thực chất là một lần xóa (`delete_delta`) cộng một lần ghi mới (`delta`); mỗi thư mục có một tệp `bucket_00000_0` khoảng 700–800 B và tệp `_orc_acid_version`. Bộ đọc hợp nhất các delta khi truy vấn, sau này compaction sẽ gộp chúng thành thư mục `base_*`.

5 Thảo luận

5.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì?

Metastore (các bảng `TBLS`, `SDS`, `PARTITIONS`… trong PostgreSQL) giữ toàn bộ siêu dữ liệu: database, tên bảng, tên và kiểu cột, SerDe, lớp định dạng vào/ra, loại bảng, `LOCATION` của bảng và từng phân vùng, giá trị phân vùng, thống kê (`numRows`, `totalSize`…) và trạng thái giao dịch ACID. HDFS chỉ chứa tệp dữ liệu. Ở Bảng 3, hai bảng ngoài chỉ là hai dòng siêu dữ liệu trỏ tới `/data/taxi_csv` và `/data/zones`.

`DROP TABLE trips_csv` (bảng ngoài) chỉ xóa siêu dữ liệu, tệp trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` thì khác: bảng này là bảng ngoài được chuyển đổi với `external.table.purge=TRUE` (Hình 7), nên lệnh xóa cả siêu dữ liệu lẫn thư mục `/user/hive/warehouse/taxi.db/trips_orc`, giống một bảng quản lý.

5.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`?

`ROW FORMAT DELIMITED` (`LazySimpleSerDe`) chỉ cắt trường theo dấu phân cách. Nó không bỏ dấu ngoặc kép và không hiểu dấu phẩy nằm trong ngoặc kép. Tệp trips sinh ra từ Parquet, không tiêu đề, không ngoặc kép, nên vậy là đủ. Tệp zones có dòng tiêu đề và giá trị như `"Allerton/Pelham Gardens"`, nên cần `OpenCSVSerde` cùng `skip.header.line.count=1`. Đổi lại, `OpenCSVSerde` coi mọi cột là `STRING` (Q2 phải viết `CAST(z.locationid AS INT)`), phân tích chậm hơn và không xử lý được giá trị NULL đặc biệt.

5.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần?

Số đo được ở Bảng 5 có hai nguyên nhân. Thứ nhất là lưu theo cột: giá trị cùng cột, cùng kiểu nằm cạnh nhau nên dictionary, RLE, delta và bit-packing nén rất tốt, nhất là với các cột như `vendorid`, `payment_type`, `ratecodeid` chỉ có vài giá trị. Thứ hai là kiểu nhị phân cộng nén khối: một timestamp chiếm 8 byte thay vì chuỗi 19 ký tự, số không còn là chuỗi chữ số kèm dấu phân cách, rồi cả khối lại được nén bằng ZLIB hoặc Snappy. ORC và Parquet chênh nhau chủ yếu do codec. Một chi tiết thú vị: `rawDataSize` so với `totalSize` của `trips_orc` (Mục 4.2) cho thấy dữ liệu khi giải nén trong bộ nhớ lớn gấp khoảng 15.7 lần trên đĩa.

5.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet?

Thời gian Q1 giảm 4.5 đến 5.3 lần (Bảng 6), và chúng tôi thấy có bốn cơ chế cùng góp phần. Trước hết là column pruning: Q1 chỉ cần 2–3 trong 19 cột, ORC và Parquet chỉ đọc đúng các *column chunk* đó, còn CSV phải đọc và phân tích cả 274 MB từng dòng một. Thứ hai là partition pruning: điều kiện lọc trên `pickup_month` khiến Hive chỉ mở thư mục `pickup_month=2024-01` (Bảng 9), trong khi bảng CSV không phân vùng nên phải quét hết rồi lọc theo timestamp. Thứ

ba, min/max và chỉ mục dòng của từng stripe/row group cho phép bỏ qua những khối không thể thỏa điều kiện. Cuối cùng, bộ đọc ORC/Parquet xử lý theo lô 1,024 dòng (`Execution mode: vectorized` ở Mục 4.5) và không mất công đổi văn bản thành số. Thí nghiệm của chúng tôi không tách riêng được phần đóng góp của từng cơ chế.

5.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`?

Phân vùng tĩnh ghi giá trị ngay trong câu lệnh, ví dụ `PARTITION (pickup_month='2024-01')`, và mọi dòng đều vào phân vùng đó. Phân vùng động lấy giá trị từ cột cuối của `SELECT`. Chế độ `strict` là mặc định để một câu truy vấn viết sai không lỡ tay tạo ra hàng nghìn phân vùng, hàng nghìn tệp và đối tượng trong Metastore. Bài này gặp đúng chuyện đó ở quy mô nhỏ: chỉ 18 dòng bất thường đã đẻ ra bốn phân vùng "lạ" (Bảng 4).

5.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao?

Theo `pulocationid` cộng ngày sẽ có khoảng $265 \times 31 \approx 8{,}000$ phân vùng mỗi tháng, mỗi phân vùng một tệp từ vài KB đến vài trăm KB. Đó là vấn đề tệp nhỏ (*small files problem*) [7], [8]. NameNode phải giữ hàng chục nghìn đối tượng trong heap. Metastore phải lưu hàng nghìn dòng `PARTITIONS`/`SDS`, và việc lập kế hoạch chậm đi vì phải liệt kê và tải siêu dữ liệu của từng phân vùng. Mỗi tệp nhỏ lại thành một split/task riêng, nên chi phí khởi tạo lấn át thời gian xử lý thật. Bốn phân vùng lẻ của `trips_orc`, mỗi cái một tệp 1,637–2,234 B, là dấu hiệu sớm của chuyện này. Phân vùng theo timestamp gốc còn tệ hơn: gần như mỗi giây một phân vùng, tức hàng triệu thư mục chứa tệp vài trăm byte. Theo chúng tôi, nên phân vùng theo cột ít giá trị mà truy vấn hay lọc, sao cho mỗi phân vùng có từ vài trăm MB đến khoảng 1 GB.

5.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định?

Partitioning tạo thư mục theo giá trị cột, hợp với cột ít giá trị mà truy vấn hay lọc (ngày, tháng, quốc gia), để bỏ qua nguyên thư mục khi đọc. Bucketing chia theo $\text{hash}(c) \bmod N$ vào N tệp, hợp với cột nhiều giá trị như mã định danh hay `pulocationid`. Nó cho phép lấy mẫu bằng `TABLESAMPLE`, bucket map join, sort-merge-bucket join, và tránh shuffle khi hai bảng được bucket giống nhau trên khóa join. Số tệp cố định vì N đã chốt trong DDL (`INTO 4 BUCKETS`): mỗi dòng luôn rơi vào bucket hash mod 4, dữ liệu nhiều hay ít cũng vậy. Kết quả ở Bảng 10 khớp với điều này. Hai kỹ thuật dùng chung được, ví dụ phân vùng theo tháng rồi trong mỗi phân vùng chia bucket theo zone.

5.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao?

18 dòng ngoài tháng 01/2024 (Hình 7) được tóm lại ở Bảng 12.

Bảng 12. Các dòng có thời gian đón khách nằm ngoài tháng 01/2024.

Phân vùng	Số dòng	Mô tả	Đánh giá
2002-12	2	2002-12-31 22:59:39, zone 170, một cặp `total_amount` +10.5/−10.5	Lỗi đồng hồ; một chuyến bị hoàn tiền
2009-01	3	Ngày 2009-01-01, cả ba trả khách tại zone 264	Lỗi đồng hồ thiết bị
2023-12	10	2023-12-31 23:39–23:58	Chuyến thật bắt đầu trước giao thừa
2024-02	3	2024-02-01 00:00:17–00:01:15	Chuyến thật bắt đầu ngay sau nửa đêm

Hai nhóm này cần xử lý khác nhau. Dòng năm 2002 và 2009 là lỗi đồng hồ, còn dòng 2023-12 và 2024-02 là chuyến thật ở biên tháng. Trong một pipeline thật, chúng tôi sẽ so thời gian sự kiện với tháng của tệp nguồn, đẩy dòng ngoài khoảng hợp lệ vào một bảng quarantine kèm lý do và cảnh báo khi số dòng này tăng. Dòng biên tháng hợp lệ thì ghi vào đúng phân vùng của nó, hoặc giữ theo tháng của tệp nguồn, tùy quy ước của nhóm. Điều quan trọng là không để phân vùng động lặng lẽ tạo phân vùng rác. Iceberg giải quyết một phần chuyện này bằng phân vùng ẩn và cho phép đổi cách phân vùng về sau.

5.9 Vì sao `count(*)` không nhanh hơn sau khi đã có thống kê?

Kết quả này làm chúng tôi bất ngờ lúc chạy. Thống kê đã `COMPLETE` nhưng `count(*)` vẫn quét bằng Tez, và thời gian xấp xỉ lần quét trước khi có thống kê (Mục 4.7). Lời giải thích hợp lý nhất là `trips_orc` thực chất là bảng EXTERNAL (bảng được chuyển đổi), và Hive không tin thống kê của bảng ngoài để trả lời thay cho việc quét, vì tệp có thể bị sửa từ bên ngoài Hive. Với bảng quản lý ACID, câu trả lời sẽ lấy thẳng từ Metastore và kế hoạch chỉ còn `Fetch`. Chúng tôi chưa kiểm chứng điều này trên `zone_notes`, nên đây vẫn là giả thuyết.

5.10 Hạn chế của thực nghiệm

Mọi thứ chạy trên một máy, Tez ở chế độ cục bộ, HDFS chỉ một DataNode với hệ số nhân bản 1, nên kết quả không phản ánh tính song song hay chi phí mạng của cụm thật. Mỗi truy vấn chỉ đo

hai lần, chưa đủ để nói về phương sai; riêng lần `count(*)` đo lại sau thống kê đã chậm hơn lần trước gần 1 s. Dữ liệu chỉ một tháng (khoảng 274 MB), nên chi phí khởi động JVM/Tez chiếm phần đáng kể trong các thời gian 1.6–2.7 s.

6 Kết luận

Metastore chỉ là một cơ sở dữ liệu quan hệ chứa lược đồ và vị trí, dữ liệu thật nằm trên HDFS. Chuyển CSV sang định dạng cột giảm kích thước 4–5 lần (Bảng 5) và giảm thời gian Q1 tương ứng (Bảng 6), nhờ cắt tỉa cột, cắt tỉa phân vùng, predicate pushdown và thực thi vector hóa. Hive chỉ cắt tỉa phân vùng khi điều kiện lọc nằm đúng trên cột phân vùng; lọc trên cột thời gian gốc thì vẫn đọc cả 5 phân vùng. Bucketing cho số tệp cố định và gom các dòng cùng khóa vào một tệp. Bảng ACID ghi thay đổi thành thư mục delta chứ không sửa tệp.

Từ bài này, chúng tôi rút ra vài điều cho thực tế. Nên lưu dữ liệu phân tích ở ORC hoặc Parquet, phân vùng theo ngày hoặc tháng và viết truy vấn lọc thẳng trên cột phân vùng. Tránh phân vùng theo cột nhiều giá trị, và kiểm tra thời gian sự kiện trước khi ghi bằng phân vùng động. Nếu có thêm thời gian, chúng tôi muốn nạp tháng thứ hai bằng phân vùng tĩnh, so sánh khi bật và tắt vector hóa, thử schema-on-read bằng cách đổi `LOCATION`, và chuyển sang Iceberg để so với phân vùng kiểu thư mục.

Tài liệu tham khảo

[1] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, R. Murthy, “Hive: A Warehousing Solution Over a Map-Reduce Framework,” *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[2] Apache Software Foundation, “Apache Hive Language Manual: DDL,” Apache Hive Wiki. https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

[3] Apache Software Foundation, “ORC Specification v1,” Apache ORC. https://orc.apache.org/specificatio

[4] Apache Software Foundation, “Apache Parquet Documentation,” Apache Parquet. https://parquet.apache.org/docs/

[5] Y. Huai, A. Chauhan, A. Gates, G. Hagleitner, E. N. Hanson, O. O’Malley, J. Pandey, Y. Yuan, R. Lee, X. Zhang, “Major Technical Advancements in Apache Hive,” in *Proc. ACM SIGMOD*, 2014, pp. 1235–1246.

[6] B. Saha, H. Shah, S. Seth, G. Vijayaraghavan, A. Murthy, C. Curino, “Apache Tez: A Unifying Framework for Modeling and Building Data Processing Applications,” in *Proc. ACM SIGMOD*, 2015, pp. 1357–1369.

[7] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O’Reilly Media, 2015.

[8] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

Phụ lục A: Danh mục tệp nộp kèm

Danh sách tệp nộp kèm ở Bảng 13.

Bảng 13. Danh mục tệp nộp kèm trong `submission/L03/`.

Tệp	Nội dung
`01_external_tables.sql`	Bảng ngoài `trips_csv`, `zones`
`02_partitioned_tables.sql`	Bảng phân vùng `trips_orc`, `trips_parquet`
`03_compare_formats.sql`	Q1 trên CSV/ORC/Parquet và Q2 nối `zones`
`04_partition_pruning.sql`	`EXPLAIN DEPENDENCY`/`EXPLAIN EXTENDED`
`05_bucketing.sql`	Bảng `trips_bucketed` 4 bucket
`01_stack_and_upload.txt`	Trạng thái container, chuyển đổi và tải CSV lên HDFS
`02_external_tables.txt`	Log chạy `01_external_tables.sql`
`03_metastore_tbls.txt`	Truy vấn `TBLS JOIN SDS` trong PostgreSQL
`04_partitioned_tables.txt`	Log nạp ORC/Parquet và `SHOW PARTITIONS`
`05_sizes_hdfs.txt`	`hdfs dfs -du` và `hdfs dfs -ls -R` của `trips_orc`
`06_describe_and_bad_rows.txt`	`DESCRIBE FORMATTED trips_orc` và 18 dòng ngoài tháng
`07_compare_formats_run1.txt`, `08_compare_formats_run2.txt`	Hai lần chạy so sánh định dạng
`09_partition_pruning.txt`	Log `EXPLAIN` cắt tỉa phân vùng
`10_bucketing.txt`	Log bucketing và danh sách tệp bucket
`11_stretch_stats_acid.txt`, `12_stretch_stats_answer.txt`	Phần mở rộng thống kê và ACID
`clean.sh`	Lọc log, chỉ giữ bảng kết quả, số dòng, thời gian và lỗi
`img/`	Ảnh chụp terminal và giao diện web

Phụ lục B: Trích đoạn mã nguồn

Trong `01_external_tables.sql`, bảng `zones` dùng `OpenCSVSerde` và bỏ dòng tiêu đề:

```
CREATE EXTERNAL TABLE zones (
  locationid STRING, borough STRING, zone STRING, service_zone STRING
```

```
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Trong `02_partitioned_tables.sql`, chúng tôi bật phân vùng động, tính `pickup_month` ở cột cuối của `SELECT` và khai báo bảng Parquet nén Snappy (câu nạp `trips_parquet` giống hệt câu nạp `trips_orc`):

```
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;

CREATE TABLE trips_parquet ( ... cùng 19 cột như trips_orc ... )
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');
```

Trong `04_partition_pruning.sql`, câu `EXPLAIN` lọc theo cột thời gian gốc để so với cách lọc theo cột phân vùng:

```
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc
WHERE tpep_pickup_datetime >= '2024-01-01' AND tpep_pickup_datetime <
 ↪  '2024-02-01';
```

Trong `05_bucketing.sql`, bảng bucketing chia theo `pulocationid` vào 4 bucket:

```
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP, pulocationid INT, dolocationid INT,
  fare_amount DOUBLE, total_amount DOUBLE
)
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;
```